

Voice-Based Concept Understanding Analyser (VBCUA)

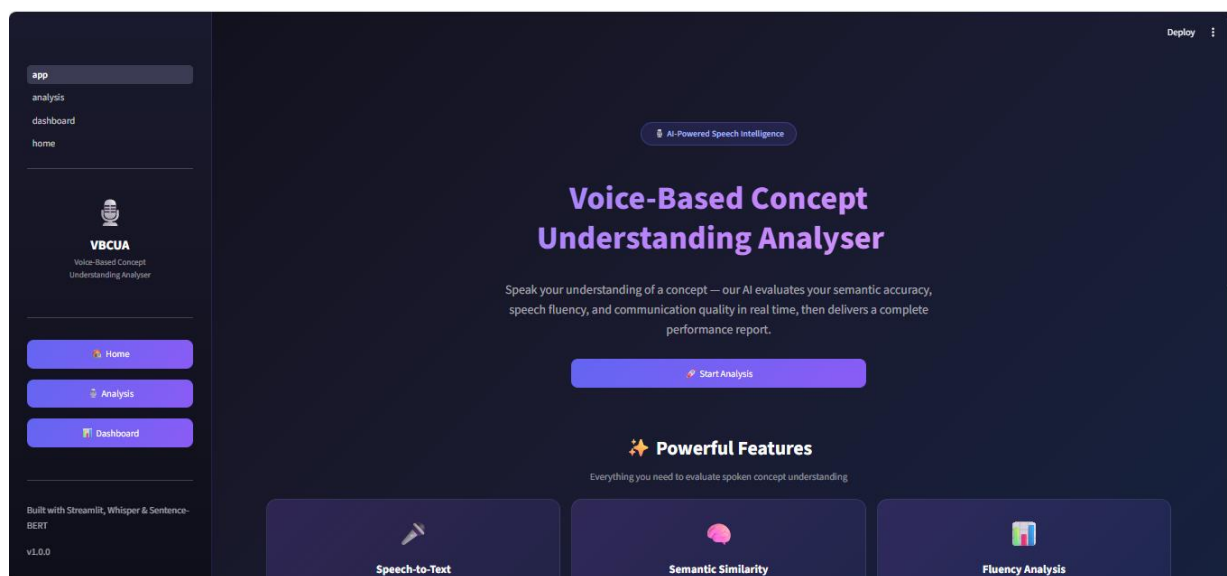
Project Description:

VBCUA is an AI-powered learning evaluation platform that analyzes a user's conceptual understanding through voice explanations. The system allows learners to upload an audio explanation of a selected concept. The uploaded speech is processed using AI models to evaluate semantic understanding, speech fluency, and communication quality.

The application uses OpenAI Whisper for speech-to-text conversion, Sentence Transformer models for semantic similarity analysis, and audio processing techniques for fluency evaluation.

The system generates performance scores, visual dashboards, personalized AI feedback, and downloadable PDF reports.

VBCUA helps students improve their learning clarity, presentation skills, and communication abilities through automated AI-based feedback.



Scenarios

Scenario 1:

A student selects Machine Learning as a concept and uploads a voice explanation.

The system converts the speech into text, compares the explanation with the reference concept, and generates understanding scores with feedback.

Scenario 2:

A learner preparing for a presentation uploads a practice explanation.

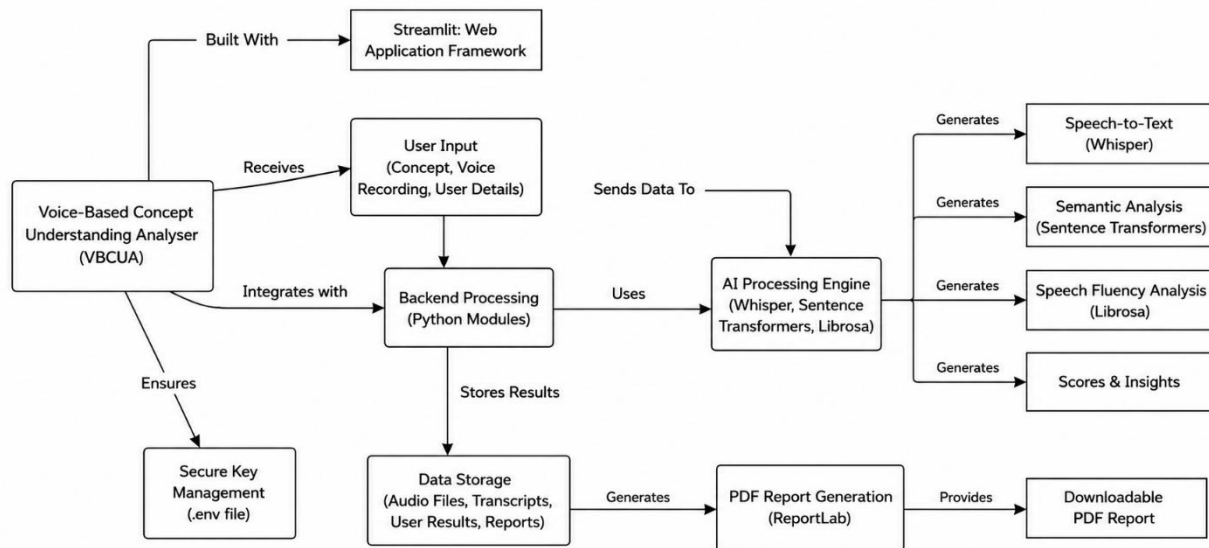
The system analyzes speech fluency, detects pauses and filler words, and provides improvement suggestions.

Scenario 3:

A user wants a complete performance summary.

The application generates a dashboard containing scores, charts, AI feedback, and a downloadable PDF report

Technical Architecture:



Description:

VBCUA follows a modular AI-based architecture. The frontend layer is developed using Streamlit, providing an interactive web interface. The backend logic is implemented using Python modules that manage audio processing, AI analysis, scoring, and report generation. The AI layer includes OpenAI Whisper for speech recognition and Sentence Transformers for concept similarity evaluation. Audio processing libraries analyze speech features, while visualization and reporting modules generate dashboards and PDF reports.

Pre-requisites:

- 📄 Streamlit → <https://docs.streamlit.io/>
- 📄 Python → <https://docs.python.org/3/>
- 📄 NLTK → <https://www.nltk.org/>
- 📄 Whisper → <https://github.com/openai/whisper>
- 📄 Sentence Transformers → <https://www.sbert.net/>
- 📄 Librosa → <https://librosa.org/doc/latest/index.html>
- 📄 Matplotlib → <https://matplotlib.org/stable/index.html>
- 📄 Git → <https://git-scm.com/doc>
- 📄 VS Code → <https://code.visualstudio.com/docs>

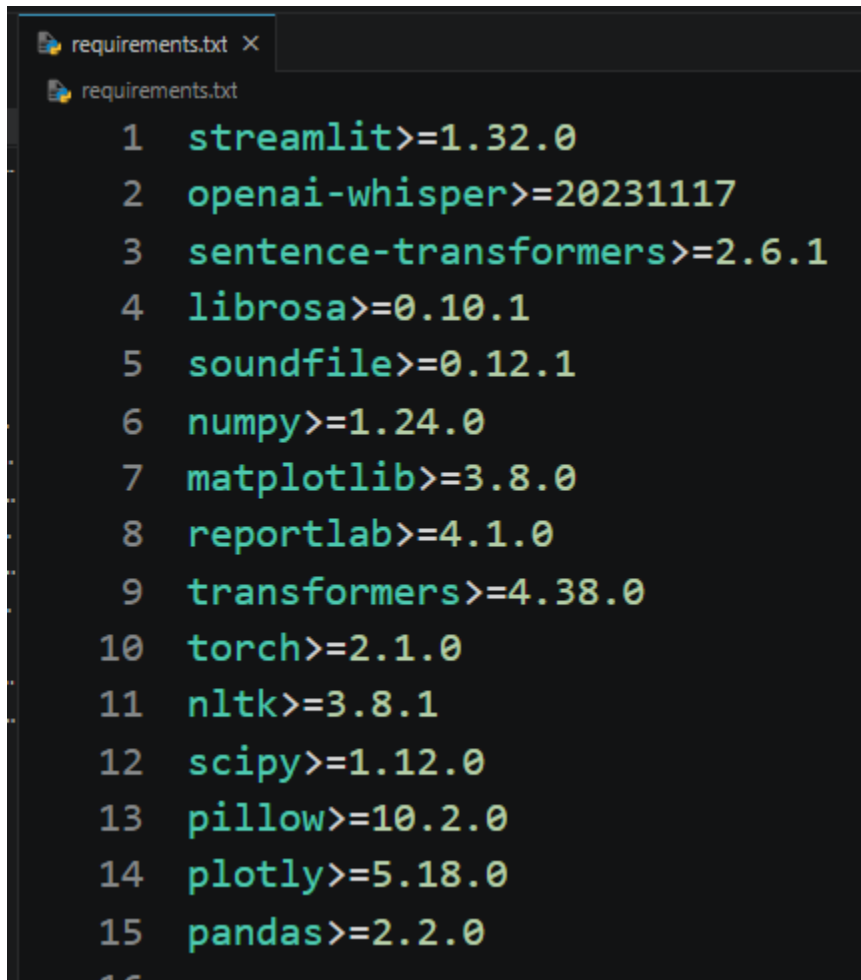
Project Workflow:

Milestone 1: Environment Setup and Project Architecture

- **Activity 1.1:**
- Install Python 3.11 and configure the development environment.

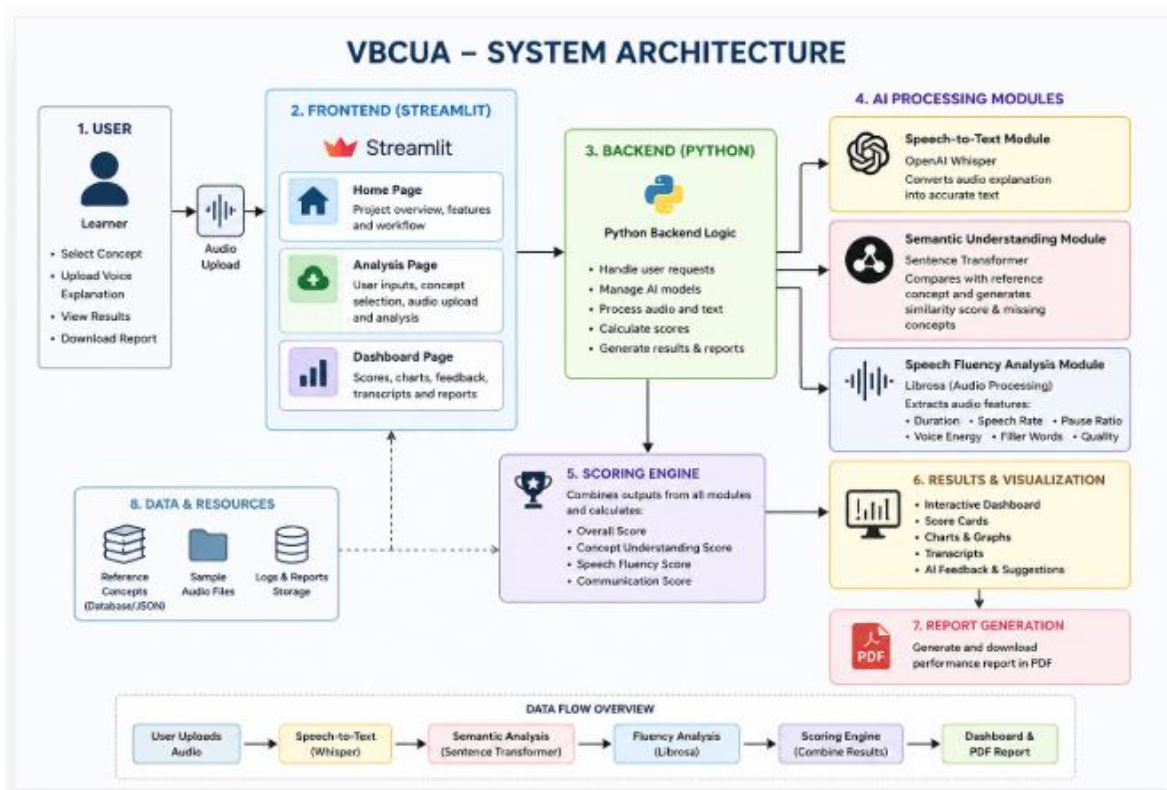
```
(vbcu_env) PS C:\Users\Dell\OneDrive\Desktop\Smartbridge\Voice-Based-Concept-Understanding-Analyser> python --version
Python 3.11.9
(vbcu_env) PS C:\Users\Dell\OneDrive\Desktop\Smartbridge\Voice-Based-Concept-Understanding-Analyser>
```

- **Activity 1.2:**
- Create a virtual environment and install required dependencies.

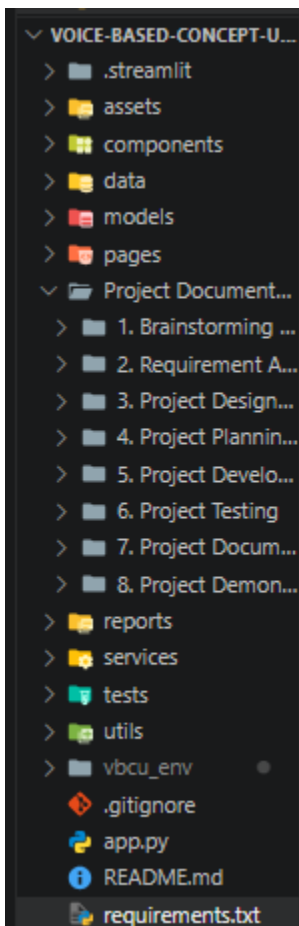


```
requirements.txt
1 streamlit>=1.32.0
2 openai-whisper>=20231117
3 sentence-transformers>=2.6.1
4 librosa>=0.10.1
5 soundfile>=0.12.1
6 numpy>=1.24.0
7 matplotlib>=3.8.0
8 reportlab>=4.1.0
9 transformers>=4.38.0
10 torch>=2.1.0
11 nltk>=3.8.1
12 scipy>=1.12.0
13 pillow>=10.2.0
14 plotly>=5.18.0
15 pandas>=2.2.0
```

- **Activity 1.3:**
- Define the architecture of the VBCUA application including frontend, AI processing modules, and report generation.



-
- **Activity 1.4:**
- Initialize Streamlit application structure and organize project folders.



Milestone 2: Core Functionalities Development

- **Activity 2.1:**
- Develop Speech-to-Text Conversion Module
- Implement OpenAI Whisper model to convert uploaded audio explanations into text format.

```
@st.cache_resource(show_spinner=False)
def load_whisper_model(model_size: str = "base"):
    """Load and cache the Whisper model."""
    try:
        import whisper
        model = whisper.load_model(model_size)
        return model
    except ImportError:
        st.error("openai-whisper is not installed. Run: pip install openai-whisper")
        return None

def transcribe_audio(audio_path: str, model_size: str = "base") -> dict:
    """
    Transcribe audio file using Whisper.

    Args:
        audio_path: Path to the audio file (.wav/.mp3/.m4a)
        model_size: Whisper model size ('tiny', 'base', 'small', 'medium', 'large')

    Returns:
        dict with keys: text, word_count, language, segments
    """
    model = load_whisper_model(model_size)
    if model is None:
        return _dummy_transcription()

    try:
        result = model.transcribe(audio_path, fp16=False)
        text = result.get("text", "").strip()
        words = [w for w in text.split() if w.strip()]
    
```

- **Activity 2.2:**
- Develop Semantic Understanding Module
- Implement Sentence Transformer model to compare user explanations with reference concepts.
- Generate similarity scores and identify missing concepts.

```
@st.cache_resource(show_spinner=False)
def load_sbert_model():
    """Load and cache the Sentence-BERT model."""
    try:
        from sentence_transformers import SentenceTransformer
        model = SentenceTransformer("all-MiniLM-L6-v2")
        return model
    except ImportError:
        st.warning("sentence-transformers not installed. Using fallback similarity.")
        return None

def compute_semantic_similarity(student_text: str, concept_name: str) -> dict:
    """
    Compute semantic similarity between student text and reference explanation.

    Args:
        student_text: Transcribed student explanation
        concept_name: Selected concept key from concepts.json

    Returns:
        dict with similarity_score (0-1), understanding_analysis, key_concept_coverage
    """
    concepts = load_concepts()
    if concept_name not in concepts:
        return {"error": f"Concept '{concept_name}' not found"}

    reference = concepts[concept_name]["reference"]
    key_concepts = concepts[concept_name].get("key_concepts", [])

    # Compute cosine similarity with SBERT
    model = load_sbert_model()
    if model is not None:
        similarity = _sbert_similarity(model, student_text, reference)
```

-
-
- **Activity 2.3:**
- **Develop Speech Fluency Analysis Module**
- Analyze audio features including:
 - Speech duration
 - Speech rate
 - Pause ratio
 - Voice energy
 - Filler words
 - Communication quality.

```
def analyze_audio(audio_path: str) -> dict:
    """
    Perform comprehensive audio analysis on a file.

    Args:
        audio_path: Path to audio file

    Returns:
        dict with waveform, duration, rms_energy, pause_ratio, speech_rate_category, confidence
    """
    try:
        import librosa
        y, sr = librosa.load(audio_path, sr=None, mono=True)
        return _compute_features(y, sr)
    except ImportError:
        return _dummy_audio_features()
    except Exception as e:
        return {"error": str(e), "success": False}

def _compute_features(y: np.ndarray, sr: int) -> dict:
    """Extract audio features from waveform array."""
    import librosa

    duration = librosa.get_duration(y=y, sr=sr)

    # RMS energy (volume indicator)
    rms = librosa.feature.rms(y=y)[0]
    rms_mean = float(np.mean(rms))
    rms_db = float(librosa.amplitude_to_db(np.array([rms_mean]))[0])

    # Silence / pause detection
    silence_threshold = rms_mean * 0.15
    is_silent = rms < silence_threshold
    pause_ratio = float(np.sum(is_silent) / len(is_silent))

    # Zero-crossing rate (clarity proxy)
    zcr = librosa.feature.zero_crossing_rate(y)[0]
```

Milestone 3: App.py Development

Milestone 3 focuses on developing the main application file app.py, which controls the complete workflow of VBCUA.

The application manages user interaction, page navigation, AI model execution, result calculation, and dashboard display.


```
import streamlit as st
from utils.helpers import inject_custom_css
from pages.home import render_home
from pages.analysis import render_analysis
from pages.dashboard import render_dashboard

def main():
    st.set_page_config(
        page_title="VBCUA – Voice-Based Concept Understanding Analyser",
        page_icon="🎧",
        layout="wide",
        initial_sidebar_state="expanded",
    )
```

Activity 3.1: Writing Main Application Logic in app.py

Define Application Pages:

Home Page:

Displays project introduction, features, and workflow.

Analysis Page:

Allows users to:

- Enter user name
- Select concept
- Upload voice recording
- Start AI analysis

Dashboard Page:

Displays generated analysis results.

Implement Processing Flow:

User Audio Upload



Whisper Speech Recognition



Semantic Similarity Analysis



Fluency Evaluation



Score Generation



Dashboard Display

Milestone 4: Frontend Development

- **Activity 4.1: Designing and Developing the User Interface**

The VBCUA user interface is developed using Streamlit with three main sections: Home, Analysis, and Dashboard. The Home page displays the project overview, features, and workflow. The Analysis page allows users to select concepts, upload voice recordings, and perform AI analysis. The Dashboard page presents scores, charts, transcripts, AI feedback, and downloadable reports in a clear visual format.

- **Activity 4.2: Creating Dynamic Result Visualization**

The VBCUA application dynamically displays AI-generated results after processing the user's voice input. The dashboard shows overall score, concept understanding, speech fluency, communication score, audio waveform, and detailed feedback. The system also provides personalized suggestions and generates downloadable PDF reports for user performance analysis.

Milestone 5: Deployment

In Milestone 5, the focus is on preparing the VBCUA application for execution and deployment. This involves setting up the project environment, installing dependencies, testing all AI modules, and running the Streamlit application successfully.

Activity 5.1: Preparing the Application for Local Deployment

The VBCUA application is prepared for local deployment by configuring the Python environment and installing all required dependencies. A virtual environment is created to manage project libraries separately, and all necessary packages including Streamlit, Whisper, Sentence Transformers, Librosa, and Report Lab are installed using the requirements.txt file.

```
PS C:\Users\Del1\OneDrive\Desktop\Smartbridge\Voice-Based-Concept-Understanding-Analyser> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Del1\OneDrive\Desktop\Smartbridge\Voice-Based-Concept-Understanding-Analyser\vbcu_env\Scripts\Activate.ps1)
(vbcu_env) PS C:\Users\Del1\OneDrive\Desktop\Smartbridge\Voice-Based-Concept-Understanding-Analyser>
```

Activity 5.2: Local Testing and Verification

The VBCUA application is tested locally using the Streamlit server to verify the complete working process. The application is executed through the terminal, and all modules including audio upload, speech-to-text conversion, semantic understanding analysis, fluency evaluation, dashboard visualization, and PDF report generation are tested successfully.

Run Command: Streamlit run app.py

After execution, the application opens in the browser using the local URL:

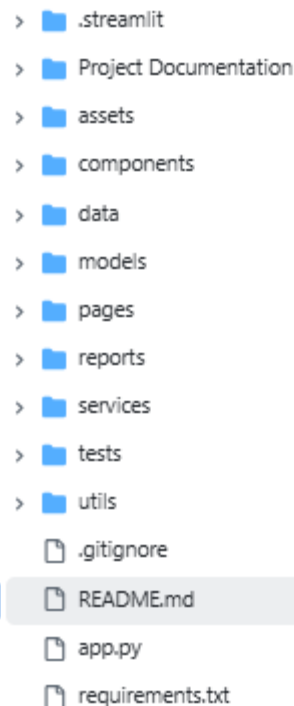
<http://localhost:8501>

Activity 5.3: Version Control and Deployment Preparation

The VBCUA project source code is managed using GitHub for version control and team collaboration. GitHub helps maintain project files, track updates, and combine contributions from different team members.

The complete application including source code, AI modules, documentation files, requirements.txt, and configuration files are uploaded into the repository.

The application can also be deployed in the future using cloud platforms such as Streamlit Cloud by connecting the GitHub repository.



```
> .streamlit
> Project Documentation
> assets
> components
> data
> models
> pages
> reports
> services
> tests
> utils
.gitignore
README.md
app.py
requirements.txt
```

Milestone 6: Conclusion

The Voice-Based Concept Understanding Analyser (VBCUA) project successfully developed an AI-powered platform for evaluating a learner's conceptual understanding through voice-based explanations.

The system integrates speech recognition, natural language processing, semantic similarity analysis, and audio feature extraction to provide a complete performance evaluation.

OpenAI Whisper is used to convert spoken explanations into text, while Sentence Transformer models analyze the similarity between user responses and reference concepts. Audio processing techniques help evaluate speech fluency, speaking speed, pauses, and communication quality.

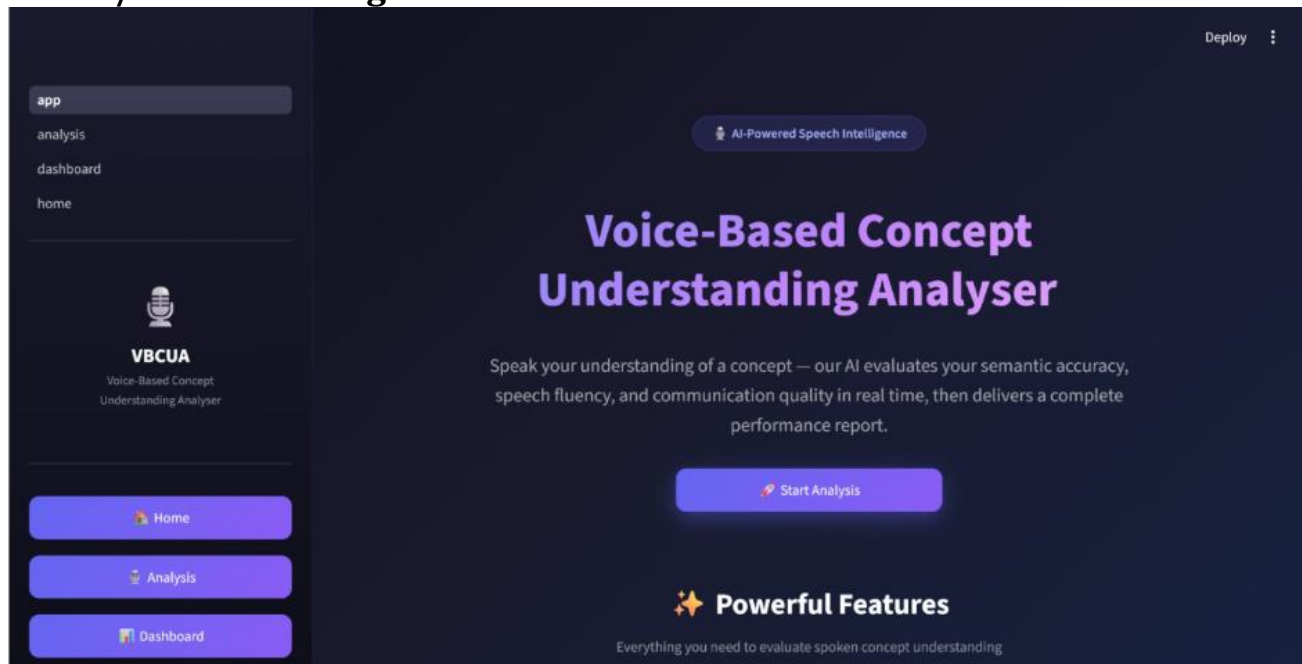
The application provides users with performance scores, interactive dashboards, AI-generated feedback, and downloadable PDF reports. This helps learners identify their strengths and improvement areas.

Overall, VBCUA demonstrates the effective use of Artificial Intelligence in improving learning assessment, presentation skills, and communication abilities.

Future improvements include real-time speech analysis, multilingual support, cloud deployment, user authentication, and personalized learning recommendation.

Exploring the Website's Web Pages:

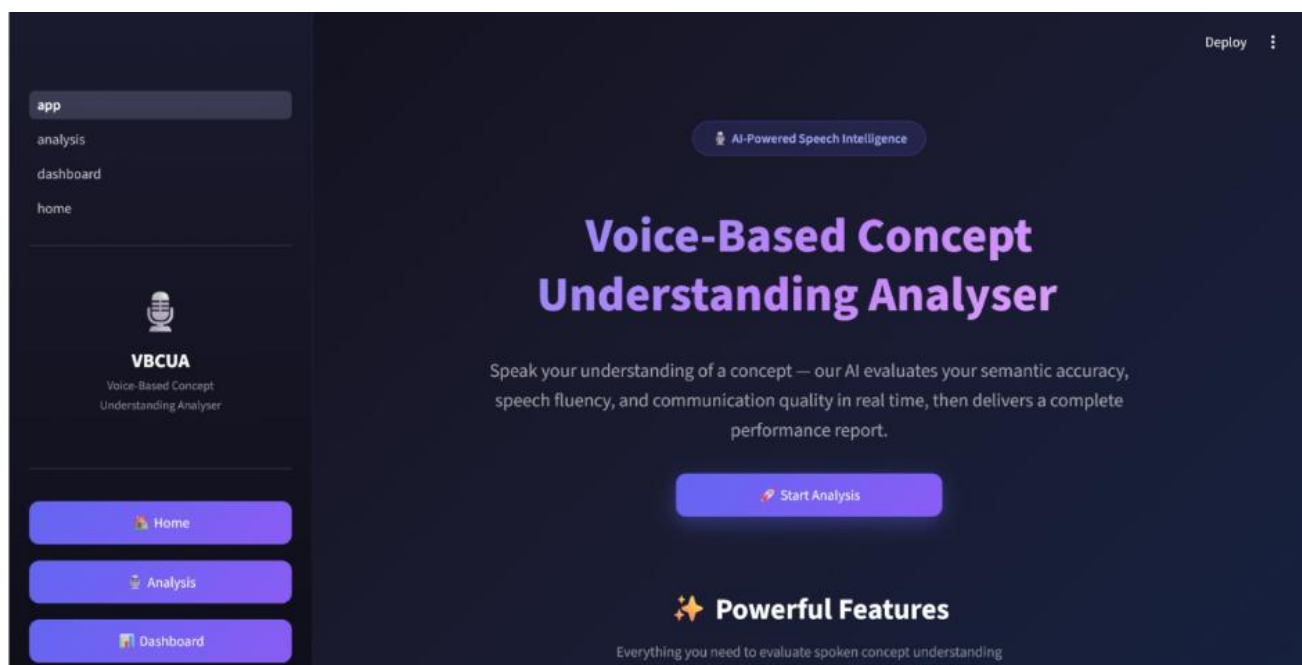
Home / Generator Page:



Description:

The Home page of Voice-Based Concept Understanding Analyser (VBCUA) serves as the landing page of the application. It provides an overview of the project purpose, which is to evaluate a learner's conceptual understanding through voice explanations. The interface is developed using Streamlit and contains the project title, introduction, navigation sidebar, feature cards, and application workflow. It highlights AI-powered features such as Speech-to-Text conversion, Semantic Similarity Analysis, Speech Fluency Evaluation, AI Scoring Engine, Dashboard Visualization, and PDF Report Generation.

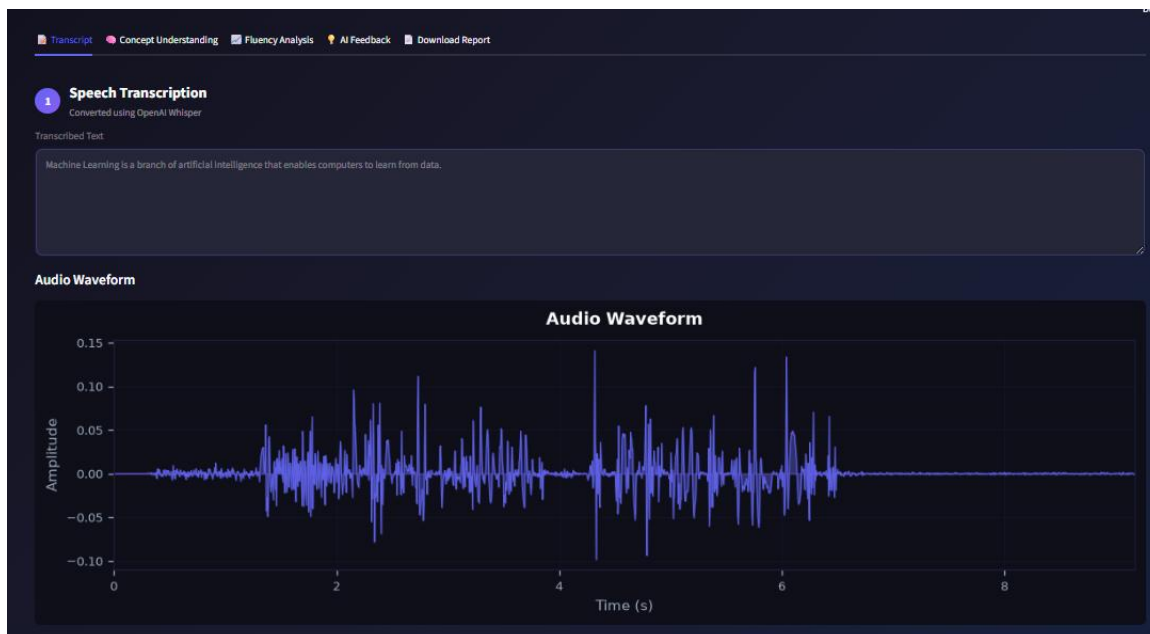
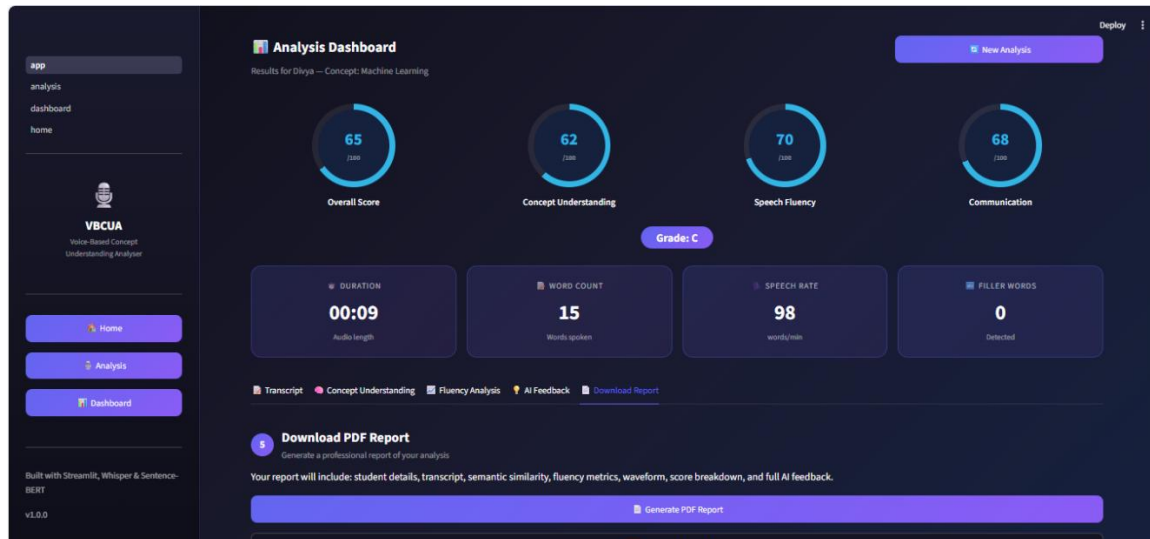
Analysis Page:

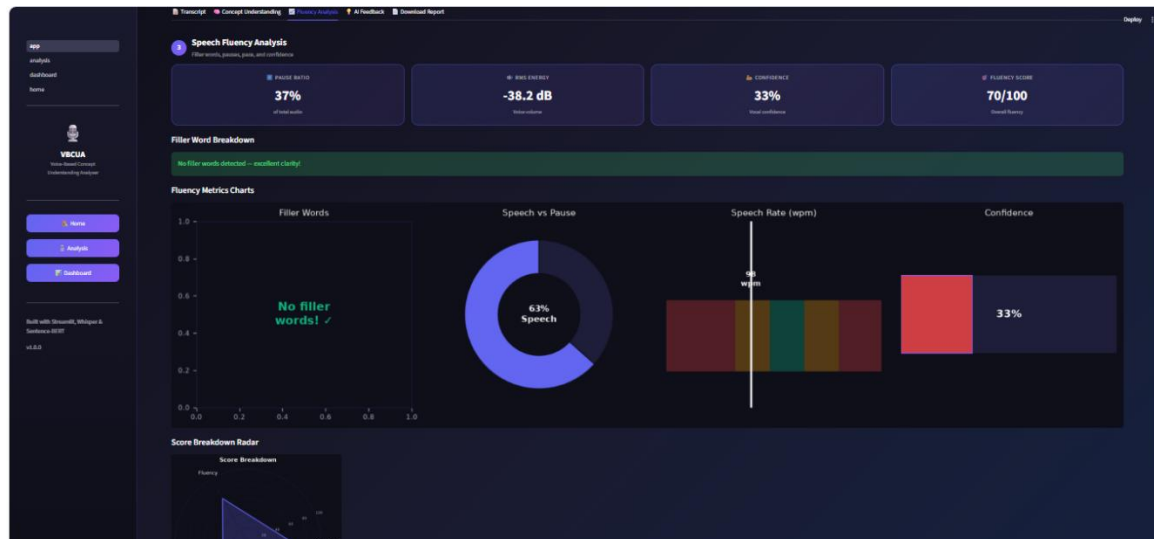


Description:

The Analysis page acts as the main input section of the VBCUA application. Users can enter their name, select a technical concept, view the reference explanation, and upload their voice explanation in audio format. After uploading the audio file, users can click the Run Full AI Analysis button to start the evaluation process. The system processes the input using OpenAI Whisper for transcription, Sentence Transformers for semantic comparison, and audio processing techniques for fluency analysis.

Dashboard Page:

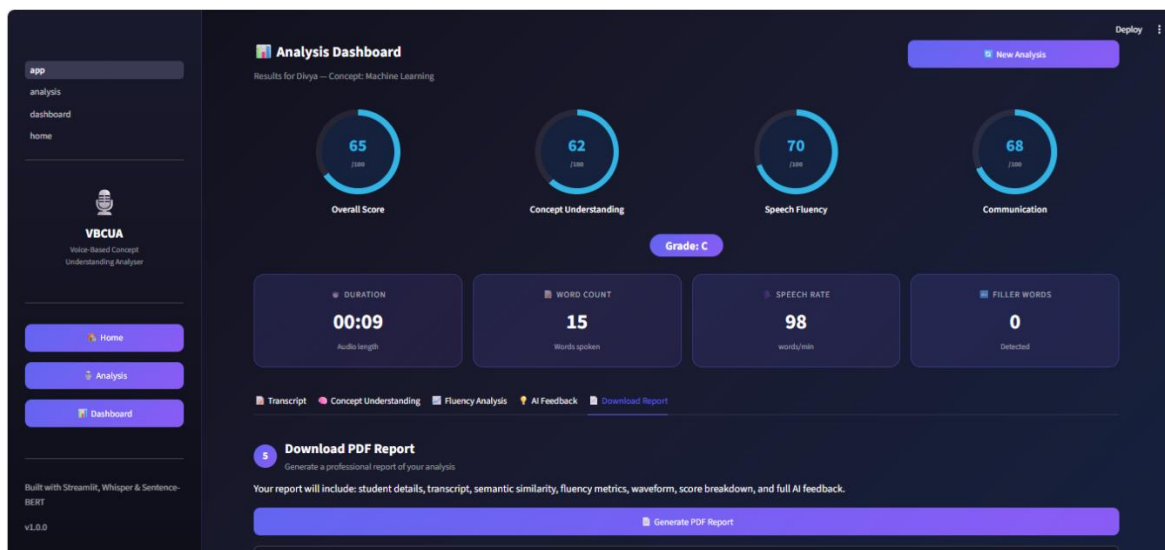




Description:

The Dashboard page dynamically presents the AI-generated analysis results after processing the uploaded voice explanation. It displays the overall performance score along with concept understanding, speech fluency, and communication quality scores. The dashboard also includes speech metrics such as duration, word count, speech rate, filler words, audio waveform visualization, transcript output, semantic understanding results, and personalized AI feedback.

Report Generation Section:



Description:

The Report Generation section allows users to create and download a complete performance analysis report. The generated PDF contains the user's transcript, understanding score, fluency evaluation, communication analysis, and AI-generated suggestions. This helps learners track their performance and identify areas for improvement.

Conclusion:

Voice-Based Concept Understanding Analyser (VBCUA) is an AI-powered learning evaluation system developed using Streamlit and Python. The application integrates OpenAI Whisper for speech recognition, Sentence Transformer models for semantic similarity analysis, and Librosa-based audio processing for speech fluency evaluation.

The system successfully analyzes voice explanations, generates understanding scores, evaluates communication quality, provides personalized feedback, and creates downloadable PDF reports. VBCUA helps students improve their conceptual clarity, presentation skills, and learning effectiveness through automated AI-based evaluation.

Future enhancements include real-time speech analysis, multilingual support, cloud deployment, user authentication, and personalized learning recommendations.